

Fintech Digital Banking System: A Full-Stack C++ and Flask Based Banking Simulation

Nigam Mehta

mehtanigam3024@gmail.com

Pursuing B.Tech in Electronics Engineering (VLSI Design and Technology)

Abstract—The application runs on a local Flask server and is accessed through a web browser via localhost for development and demonstration. This report presents a full-stack Fintech Digital Banking System combining a C++ banking core, a Python Flask web server, and a responsive browser interface. The system models account creation, balance enquiry, fund transfer, bill payment, UPI transactions, debit and credit card management, loans, fixed deposits, cheque-book management, transaction reports, analytics, and PDF statement generation. Flask communicates with the compiled C++ executable through interprocess communication using standard input and output pipes, while application state is persisted in JSON files. The frontend uses HTML, Tailwind CSS, and Vanilla JavaScript. Mathematical models for loan EMI and fixed-deposit maturity are included, and the report discusses the software architecture, data structures, API design, security considerations, testing, limitations, and future scope. Supplied screenshots of the implemented application are incorporated to document the major workflows.

Index Terms—Digital banking, C++, Flask, IPC, UPI, fixed deposit, loan EMI, debit card, credit card, PDF statement, fintech.

I. INTRODUCTION

Digital banking applications integrate multiple financial services into a single user interface. Even an academic banking application must coordinate account information, balances, transactions, payment instruments, loans, investments, and reporting while keeping the browser interface synchronized with its business logic.

The Fintech Digital Banking System follows a hybrid architecture in which the core banking operations are implemented in C++, while Flask provides the web server and API layer. The design demonstrates how a structured command-line banking engine can be exposed through a modern web interface using interprocess communication.

The system is intended for project and development purposes only. Financial operations are simulated and local JSON persistence is used instead of a production banking database.

II. PROBLEM STATEMENT

A console-only banking program is difficult to use through a modern browser and provides limited visualization. The project therefore aims to preserve modular C++ business logic while exposing banking services through Flask APIs and an interactive web interface.

The system supports account creation, balance enquiry, transfers, bill payments, UPI, debit and credit cards, loans, fixed deposits, cheque books, statements, analytics, and profile management.

III. OBJECTIVES

- 1) Develop a modular C++ banking core.
- 2) Integrate the core with a Flask web application.
- 3) Provide a responsive browser-based banking dashboard.
- 4) Implement account and transaction management.
- 5) Support transfers, bills, UPI, debit cards, and credit cards.
- 6) Implement loan EMI and fixed-deposit maturity calculations.
- 7) Generate PDF account statements.
- 8) Provide transaction analytics.
- 9) Demonstrate data structures, searching, and modular software design.

IV. SYSTEM ARCHITECTURE

The system contains three logical layers.

Frontend: HTML5, Tailwind CSS, and Vanilla JavaScript provide login, account creation, dashboard, UPI, loan, fixed-deposit, services, transactions, and card interfaces.

Middleware: Flask serves the web pages and REST-style API endpoints. It validates requests, manages user-profile information, invokes the C++ process, parses responses, generates PDF statements, and calculates analytics.

Backend Core: C++ modules include `account.cpp`, `credit.cpp`, `debit.cpp`, `fd.cpp`, `loan.cpp`, `upi.cpp`, `cheque.cpp`, `report.cpp`, `utils.cpp`, and `globals.cpp`. Shared structures and declarations are defined in `bank.h`.

The overall communication path is

Browser → Flask API → IPC → C++ Core → Persistent State.
(1)

V. TECHNOLOGY STACK

VI. CORE DATA STRUCTURES

The `Account` structure stores account number, customer name, balance, loan amount, fixed deposit, transaction history, cheque-book information, debit-card information, and credit-card information. Transactions are represented using a linked list:

```
struct Transaction {
    string type;
    double amount;
    Transaction* next;
};
```

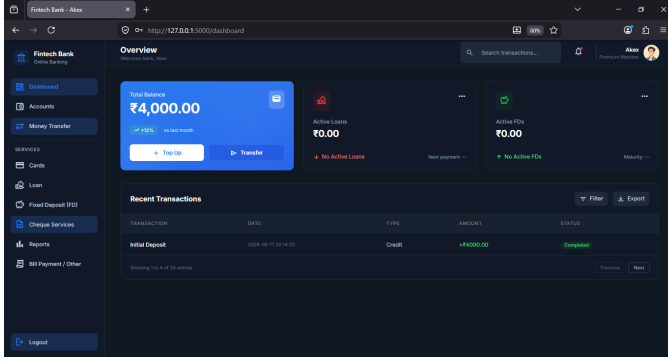


Fig. 1. Fintech Bank dashboard with integrated banking services.

TABLE I
TECHNOLOGY STACK

Layer	Technology
Frontend	HTML5, Vanilla CSS, Tailwind CSS, JavaScript
Web server	Python Flask
Backend	C++
IPC	Standard input/output process pipes
Persistence	JSON files
PDF	ReportLab
HTML processing	BeautifulSoup4
Build	CMake / g++
Client state	Browser session storage

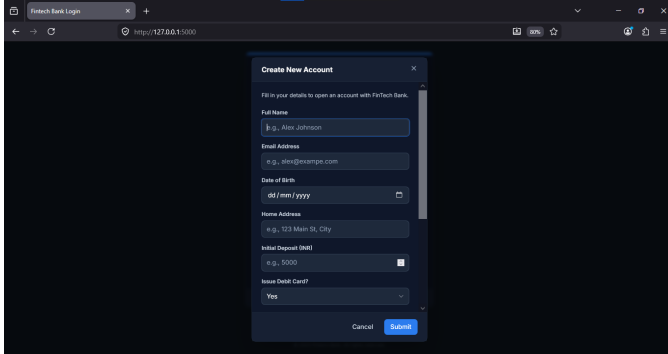


Fig. 2. Account creation interface.

Accounts are maintained using an unordered map, while a sorted vector of account numbers is available for binary search. If there are n sorted account numbers, binary search has

$$T(n) = O(\log n). \quad (2)$$

VII. ACCOUNT MANAGEMENT

Account creation automatically generates an account number starting from the configured sequence. The customer enters a name and initial deposit. Loan and fixed-deposit values are initialized, and transaction history is prepared. A debit card can optionally be issued.

Balance changes follow the general model

$$B_{k+1} = B_k + D_k - W_k, \quad (3)$$

where D_k and W_k denote deposits and withdrawals/debits.

VIII. FUND TRANSFER AND PAYMENTS

Fund transfer moves a specified amount from a source account to a destination account. The intended balance transition is

$$B'_s = B_s - A, \quad B'_d = B_d + A, \quad (4)$$

where A is the transfer amount.

Bill payment functionality supports simulated Netbanking, debit-card, and UPI payment methods. The Flask API validates the selected method and required fields before processing the operation.

IX. UPI ENGINE

The UPI module creates UPI identifiers and processes UPI payments. A mapping between UPI IDs and account numbers is maintained:

$$\text{UPI ID} \rightarrow \text{Account Number}. \quad (5)$$

This enables a UPI payment request to resolve its destination account before the simulated transfer.

X. DEBIT AND CREDIT CARDS

Debit-card functionality includes payment, blocking, and re-issue. The account model stores card number, name, expiry, CVV, and PIN. Credit-card functionality checks eligibility and stores a credit-card number and configured limit.

For a real financial application, these values would require secure tokenization, encryption, strict access controls, and compliance with applicable payment-card standards.

XI. LOAN MANAGEMENT

The loan module supports application and repayment and the web UI includes an interactive calculator. For principal P , periodic interest rate r , and n payments, the standard amortized EMI is

$$EMI = P \frac{r(1+r)^n}{(1+r)^n - 1}. \quad (6)$$

For an annual rate R and monthly payments,

$$r = \frac{R}{12 \times 100}. \quad (7)$$

Total repayment and interest are approximately

$$T = EMI \times n, \quad I = T - P. \quad (8)$$

XII. FIXED DEPOSIT MODULE

The fixed-deposit module supports maturity calculation, creation, and withdrawal. A standard compound-interest model is

$$A = P \left(1 + \frac{R}{100n}\right)^{nt}, \quad (9)$$

where P is principal, R is annual interest rate, n is compounding frequency, and t is time in years. Maturity interest is

$$I_{FD} = A - P. \quad (10)$$

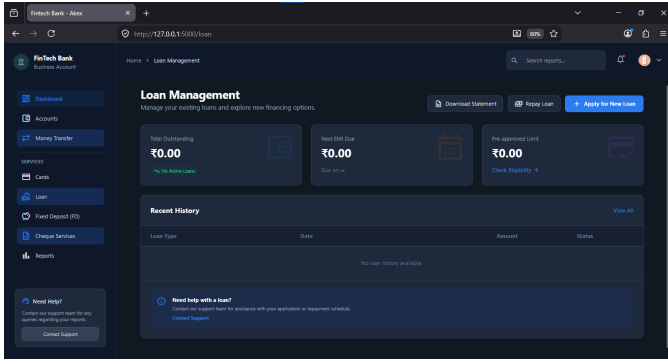


Fig. 3. Loan management and EMI calculator interface.

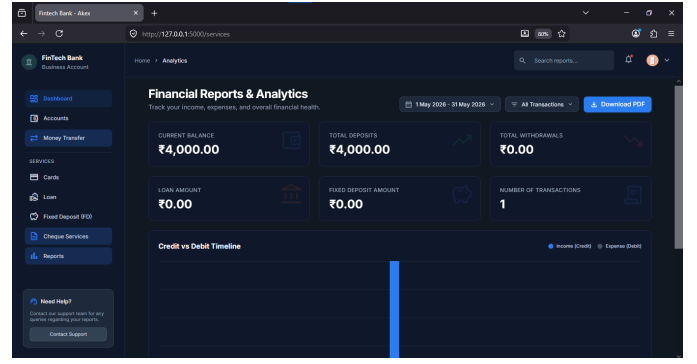


Fig. 5. Account report and statement interface.

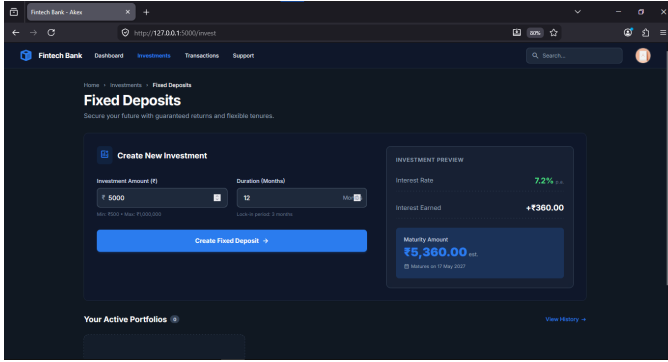


Fig. 4. Fixed-deposit and investment interface.

TABLE II
REPRESENTATIVE API ENDPOINTS

Endpoint	Purpose
POST /api/login	Authenticate user
POST /api/create_account	Create account/profile
GET /api/account/{acc_no}	Retrieve account
POST /api/transfer	Transfer funds
POST /api/pay_bill	Bill payment
POST /api/debit_payment	Debit-card payment
POST /api/create_fd	Create FD
POST /api/withdraw_fd	Withdraw FD
POST /api/apply_loan	Apply for loan
POST /api/repay_loan	Repay loan
GET /api/statement/{acc_no}	Generate statement
GET /api/analytics/{acc_no}	Analytics
POST /api/apply_cc	Credit-card application
POST /api/create_upi	Create UPI ID

XIII. CHEQUE-BOOK MANAGEMENT

Cheque-book operations are implemented as a dedicated module. Each account stores whether a cheque book exists, the starting cheque number, and the number of used cheques. A global sequence is used to generate cheque ranges.

XIV. PDF STATEMENT GENERATION

Flask uses ReportLab to generate professional PDF account statements. The report includes customer/account information, transaction history, debit and credit totals, and closing balance.

For transaction amounts x_i ,

$$D = \sum_{x_i < 0} |x_i|, \quad C = \sum_{x_i > 0} x_i. \quad (11)$$

The opening balance can be reconstructed using

$$B_{open} = B_{close} + D - C. \quad (12)$$

XV. ANALYTICS

The application provides an analytics endpoint for an account. Transaction history can be aggregated into daily deposits, withdrawals, and spending values. A cumulative balance trend can be represented as

$$B_j = B_{j-1} + d_j, \quad (13)$$

where d_j is the net transaction value for day j .

XVI. FLASK API DESIGN

Representative endpoints are shown in Table II.

XVII. INTERPROCESS COMMUNICATION

The Flask controller writes menu choices and inputs to the C++ process through standard input and reads output until the expected prompt is reached:

HTTP Request $\rightarrow$ Flask Handler $\rightarrow$ *stdin* $\rightarrow$ C++ Menu $\rightarrow$ *stdout* $\rightarrow$ Re (14)

The process controller uses a lock around communication so multiple requests do not simultaneously write to the same process input stream. If the process terminates, the controller can restart it.

This approach is straightforward for an academic project, although production systems would normally use an in-process service interface, database-backed service layer, or dedicated microservice rather than a prompt-driven executable.

XVIII. PERSISTENCE

The application persists bank and user-profile state in JSON files. Profile saving uses a temporary file followed by replacement:

```
tmp_path = PROFILES_JSON + ".tmp"
json.dump(data, f, indent=4)
os.replace(tmp_path, PROFILES_JSON)
```

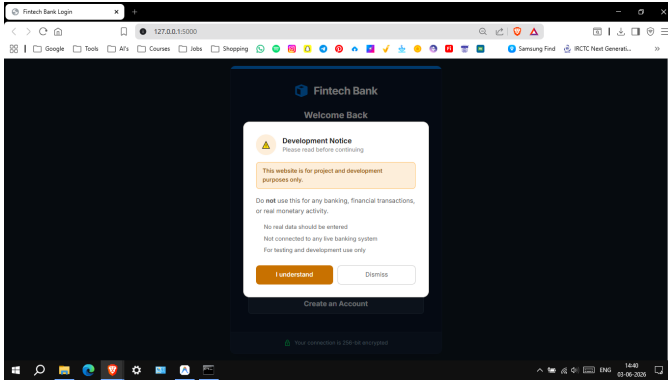


Fig. 6. Development-use warning displayed by the application.

This replacement pattern reduces the risk of leaving a partially written profile file.

XIX. SECURITY CONSIDERATIONS

The project contains a login workflow, profile management, and simulated card/payment functionality. However, it is explicitly a development simulation and should not be used with real banking credentials or money.

A production implementation would require strong password hashing, HTTPS/TLS, secure sessions, encrypted sensitive data, transactional database operations, authorization controls, audit logs, secret management, CSRF/XSS/injection protection, payment-provider verification, fraud monitoring, and secure handling of card data.

XX. FRONTEND DESIGN

The frontend is split into views for authentication, dashboard, account/profile settings, UPI, loans, fixed deposits, billing and analytics, transactions, and cards. JavaScript binds UI components to Flask APIs and maintains browser-side session state.

The dashboard consolidates balances and services so that users can move between financial operations without leaving the application.

XXI. TESTING AND RESULTS

Representative functional tests are listed in Table III. The supplied screenshots demonstrate the implemented interfaces for account creation, dashboard, loans, fixed deposits, reports, login, and development warnings.

XXII. LIMITATIONS

- 1) JSON storage is not appropriate for high-concurrency financial transactions.
- 2) The process-pipe interface is coupled to console prompts.
- 3) The project is designed for local development and demonstration.
- 4) Financial operations are simulated and are not connected to real payment networks.
- 5) Sensitive card information requires substantially stronger protection in real deployments.

TABLE III
REPRESENTATIVE TEST CASES

Test	Expected	Implemented
Create account	Unique account	Yes
Balance enquiry	Current balance	Yes
Transfer	Both balances up-dated	Yes
UPI payment	Debit/credit recorded	Yes
Loan application	Loan stored	Yes
Loan repayment	Loan reduced	Yes
FD creation	Investment recorded	Yes
FD withdrawal	Funds returned	Yes
Debit-card payment	Debit recorded	Yes
Credit-card application	Eligibility checked	Yes
Statement generation	PDF produced	Yes
Analytics	Transaction trends	Yes
Card block	Card disabled	Yes

- 6) Distributed locking and multi-instance coordination are not provided.
- 7) Authentication and authorization require hardening for production use.

XXIII. FUTURE SCOPE

Future improvements include migration to PostgreSQL or another transactional database, replacement of prompt-driven IPC with a clean service interface, real payment/UPI integration, multi-factor authentication, role-based access control, transaction auditing, distributed caching/rate limiting, HTTPS deployment, automated testing, and advanced fraud/anomaly detection.

XXIV. CONCLUSION

The Fintech Digital Banking System demonstrates an end-to-end academic banking application using a hybrid C++ and Python architecture. The C++ core provides modular account, transfer, loan, fixed-deposit, UPI, card, cheque, and reporting functionality, while Flask exposes these services through a web interface. The project additionally provides analytics, profile management, PDF statements, and a dashboard.

The system is useful as a software-engineering demonstration because it combines C++ programming, data structures, financial calculations, process communication, web APIs, frontend development, persistence, and document generation. Its development-oriented design also makes the security, database, concurrency, and deployment limitations explicit, providing clear directions for future work.

ACKNOWLEDGMENT

The author acknowledges the academic project environment and the open-source technologies used in developing the Fintech Digital Banking System.

REFERENCES

- [1] Flask Documentation, “Flask Web Development,” available at <https://flask.palletsprojects.com/>.
- [2] B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley.
- [3] ReportLab Documentation, “ReportLab User Guide,” available at <https://docs.reportlab.com/>.
- [4] Tailwind CSS Documentation, available at <https://tailwindcss.com/docs>.
- [5] Beautiful Soup Documentation, available at <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.